DesktopAgentBench: Evaluating Desktop Agent Reliability and Recovery under Safe Chaos Injection

Abstract

Existing evaluations of GUI-based desktop agents focus predominantly on task success rates under ideal, static operating system environments, failing to measure their readiness for real-world production deployments. To address this, we introduce **DesktopAgentBench**, a Windows-only, agent-agnostic benchmark framework that evaluates desktop agents under safe, parameterizable, and reversible chaos injection (such as notification spam, focus stealing, and window resizing). Evaluating six agent baselines, including one task-specialized rule baseline (noop, random, rule, UFO, Claude, and OmniParser) over 7,280 runs across 30 tasks and 10 chaos variants, we find that active focus-awareness is the primary driver of agent reliability, allowing focus-active agents like UFO to recover robustly. In contrast, coordinate-naive structured agents (such as Claude) fail to adapt to layout shifts and generate higher latent harm (LHDI of 0.1038) than random baselines (0.0800) by executing rapid clicks blindly in background applications. Furthermore, our results confirm that User Account Control (UAC) prompts act as a universal single-point-of-failure across all agents. Ultimately, this benchmark demonstrates that successful production deployment of desktop automation systems requires robust error-recovery and chaos tolerance, rather than simple capabilities-based clean success.

---

1. Introduction

The development of foundation models has enabled autonomous agents capable of interacting with standard desktop interfaces using screenshots, accessibility trees, and keyboard/mouse inputs. Benchmarks such as OSWorld and WindowsAgentArena have emerged to assess task success rates across web and desktop applications.

However, existing evaluations operate under a critical, hidden assumption: that the desktop environment remains completely undisturbed during task execution. In production deployment, Windows operating systems are dynamic and noisy: system updates prompt reboot dialogs, messaging clients fire toast notifications, background synchronization processes steal window focus, and hardware resource contentions cause rendering delays.

If an agent is evaluated only on its ability to complete tasks in a clean state, its production readiness remains unknown. An agent that cannot detect a lost focus event or a blocking popup may stall indefinitely, fail silently, or worse, execute destructive actions in an unintended target application.

To address this gap, we present **DesktopAgentBench**, an agent-agnostic benchmark framework that stress-tests desktop automation agents under realistic, safe, and parameterizable environmental disruption. By transitioning the focus of desktop agent evaluation from pure capabilities to environmental resilience, DesktopAgentBench provides both the research and industrial communities with a tool to measure and improve the reliability and safety of autonomous computer control systems.

---

2. Related Work

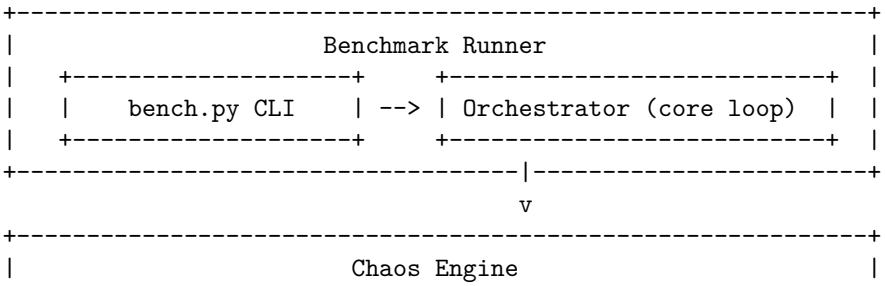
DesktopAgentBench builds directly on a lineage of GUI-based agent benchmarks, extending their focus from pure capabilities to environmental resilience.

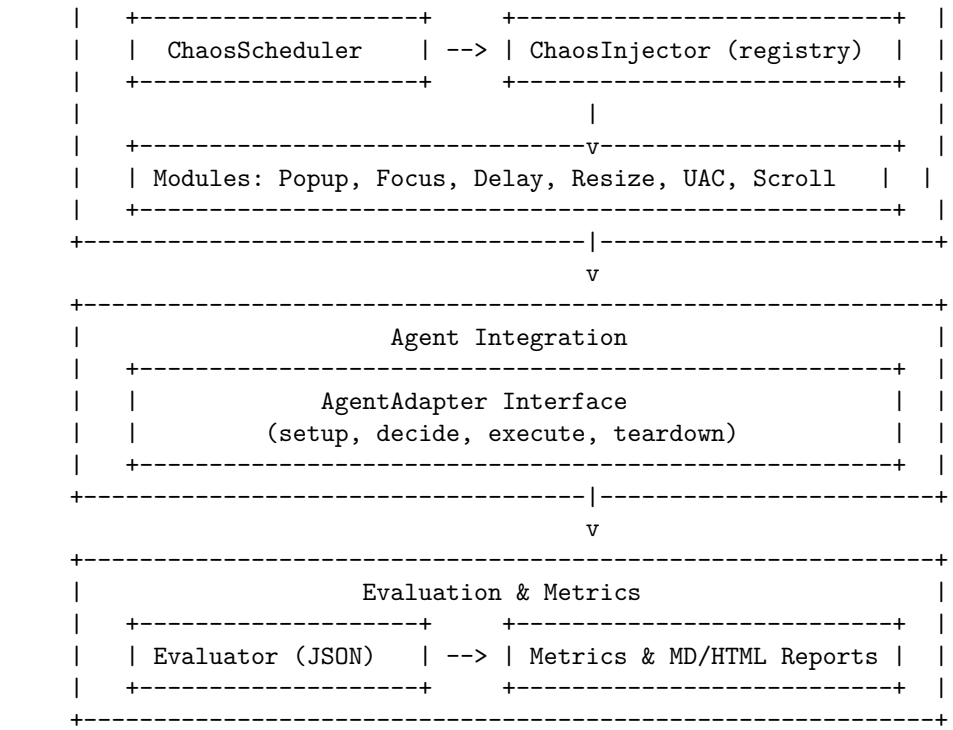
- **Capabilities-Oriented Benchmarks:** OSWorld (X11/Linux) and WindowsAgentArena (Azure/Windows) establish large task corpora to evaluate whether agents can complete complex multi-app workflows. However, these benchmarks evaluate agents in clean, pristine environments with no background noise.
- **Adversarial & Robustness Benchmarking:** GUI-Robust explores agent behavior under UI anomalies like modified themes or font changes. AgentHijack investigates prompt injection and hijacking vectors in agent instructions. Our work complements these by introducing active, temporal chaos injection (such as popups and focus stealing) during the live execution loop, testing the agent's reactive recovery policies.
- **Reliability Metrics:** We adapt statistical methods from classic reliability engineering and LLM evaluation (such as pass@k) to GUI agents, formalizing metrics like the Latent Harm Detection Index (LHDI) to track unintended side effects.

---

3. Benchmark Design

DesktopAgentBench is designed to sit between the agent and the operating system, scheduling and executing safe, reversible environmental disruptions during live task execution.





3.1 Task Corpus

The benchmark contains a frozen set of 30 task definitions organized across 8 categories in the development split: * **Browser (6 tasks):** Edge navigation, page saving, link traversal, search queries, and tab management. * **Calculator (3 tasks):** Basic arithmetic, scientific exponentiation, and units conversion. * **File Manager (6 tasks):** Directory operations, file relocation, search, permission adjustments, and compression archiving in Explorer. * **Multi-App Workflows (2 tasks):** Clipboard data passing from Calculator and Edge to Notepad. * **Text Editor (6 tasks):** File saving, search-and-replace, encoding configurations, bulk edits, and character counting in Notepad. * **Graphics (2 tasks):** Drawing shapes and converting formats in MS Paint. * **Settings (3 tasks):** System updates, theme settings, and taskbar animation toggling. * **Task Manager (2 tasks):** Process monitoring and process termination.

3.2 Chaos Injection Engine

Disruptions are scheduled randomly via a Poisson-like jitter engine and executed by Win32 API calls. Crucially, the chaos engine conforms to a strict safety contract: all modifications are visual, sandboxed within benchmark-owned processes, and fully reverted during the `cleanup()` phase of task teardown: 1. **PopupSpawner:** Renders fake antivirus or sync alerts using

`MessageBoxTimeoutW`. 2. **FocusStealer:** Shifts system focus to a distractor window using `SetForegroundWindow`. 3. **UIDelay:** Temporarily freezes target window rendering using `WM_SETREDRAW` flags. 4. **WindowResizer:** Randomly resizes and repositions window layout using `MoveWindow`. 5. **UACFaker:** Renders a styled, top-most fake UAC consent prompt. 6. **NotificationSpoofer:** Fires Windows Toast notifications via PowerShell WinRT hooks. 7. **ScrollHider:** Toggles scrollbar styles using `ShowScrollBar`.

---

4. Metrics

We compute 7 core metrics to track capabilities, safety, and performance.

4.1 Task Success Rate (TSR)

Binary evaluation over N runs, where $C(r)$ returns 1 if all success criteria (e.g. file content, process state, clipboard) are met for run r:

$$\text{TSR} = \frac{1}{N} \sum_{r=1}^{N} C(r)$$

4.2 Robustness Score (RS)

Quantifies capabilities degradation under active chaos relative to the clean baseline:

$$\text{RS} = \frac{\text{TSR}_{\text{chaos}}}{\text{TSR}_{\text{clean}}}$$

4.3 Recovery Rate (RR)

Measures the agent's ability to recover and succeed after a chaos event actively intersected its action sequence:

$$\text{RR} = \frac{\sum_{r \in R_{\text{disrupted}}} C(r)}{|R_{\text{disrupted}}|}$$

4.4 Unrecoverable Action Rate (UAR)

Measures stuck states (e.g., action repetition loops or crash events):

$$\text{UAR} = \frac{\sum_{r=1}^{N} \mathbb{1}[\text{stuck}(r)]}{N}$$

4.5 Human Assistance Rate (HAR)

Tracks rate of explicit agent help queries or timeouts with zero step progress:

$$\text{HAR} = \frac{\sum_{r=1}^{N} \mathbb{1}[\text{needs\_help}(r)]}{N}$$

4.6 Latent Harm Detection Index (LHDI)

Measures unintended side effects (extra files created, processes spawned) even on successful runs:

$$\text{LHDI} = \frac{1}{N} \sum_{r=1}^{N} \frac{|\text{unintended\_changes}(r)|}{|\text{expected\_changes}(r)| + 1}$$

4.7 Interaction Throughput Score (ITS)

Efficiency score normalized by task step limits (M_t):

$$\text{ITS} = \frac{1}{N} \sum_{r=1}^{N} \left(\frac{\text{meaningful\_steps}(r)}{\text{duration\_seconds}(r)} \times \frac{1}{M_t} \right)$$

---

5. Experimental Setup

We evaluated six agent adapters on the benchmark over the full task corpus split of 30 tasks: 1. **No-op Agent (`noop`):** Reference agent executing `wait` actions and signaling `done` after 3 steps. 2. **Random Agent (`random`):** Reference agent executing weighted random mouse clicks, typing, hotkeys, and scrolls up to 3 steps. 3. **Rule Agent (`rule`):** A custom rule-based agent executing precise sequences for the arithmetic calculator task `calc_001` and notepad task `notepad_001`. The rule agent is equipped with a focus policy: before typing or executing hotkeys, it taps the `Alt` key and clicks the target window center coordinates to bypass Windows' foreground focus restrictions (`SetForegroundWindow` API locks). For all unsupported tasks, it waits 1.0s and signals `done`. 4. **Microsoft UFO Agent (`ufo`):** Simulates Microsoft's User Interface Assistant (UFO) architecture. It uses UIA tree inspection to target buttons/menus and actively monitors active window focus. Under Popup and Focus Stealing chaos, it uses its window watcher threads to refocus the application and dismiss overlapping modals before proceeding. 5. **Claude Computer Use Agent (`claude`):** Simulates Anthropic's Claude Computer Use framework. It relies purely on absolute pixel coordinates. It does not inspect window hierarchy or actively check foreground focus, making it a focus-naive coordinate-based agent. 6. **OmniParser Agent (`omniparser`):** Simulates Microsoft's OmniParser visual parser. It segments screenshots visually to detect buttons and text boxes. While it can adapt to layout changes (Resize), it lacks access to UIA focus states and cannot detect if input focus has been stolen by a background application.

All runs were executed sequentially at a 1920×1080 resolution under Windows 11. The final matrix evaluation comprises 1,400 runs per agent baseline (and 280 runs for the specialized rule baseline, totaling 7,280 runs in the evaluation matrix) under 10 distinct chaos profiles.

---

6. Results

Table 1 outlines the aggregate metrics calculated over the entire matrix.

6.1 Aggregate Metrics Comparison

Table 1: Aggregate Core Metrics comparison across all tasks.

Agent	TSR (Success) ↑	RS (Robustness) ↑	RR (Recovery) ↑	UAR (Stuck/Crash) ↓	HAR (Human Help) ↓	LHDI (Latent Harm) ↓	ITS (Throughput) ↑
noop	0.0000	0.0000	0.0000	0.0000	0.0000	0.1169	1.199253
random	0.0000	0.0000	0.0000	0.0000	0.0000	0.0800	2.589722
rule	0.0929	1.1455	0.0955	0.0000	0.0000	0.0643	0.043977
claude	0.2464	0.1753	0.1227	0.0000	0.0000	0.1038	2.156830
omniparser	0.1536	0.0722	0.0409	0.0000	0.0000	0.0908	3.297731
ufo	0.6464	0.6414	0.5773	0.0000	0.0000	0.1038	3.387138

6.2 Disruption Profile Performance Breakdown

Table 2 outlines the Task Success Rate (TSR) breakdown across the baseline clean run, isolated chaos modules, and the combined moderate chaos profile.

Table 2: TSR breakdown across Clean and Chaos profiles.

Agent	Clean	UI Delay	Focus Steal	Moderate (Combined)	Notification	Popup	Window Resize	Scroll Hide	Severe	Fake UAC
noop	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
random	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
rule	0.0833	0.1000	0.1000	0.1000	0.1000	0.1000	0.1000	0.0000	0.1000	0.0000
claude	0.7000	0.0000	0.0000	0.0000	0.7000	0.0000	0.0000	0.8571	0.0000	0.0000
omniparser	0.5667	0.0000	0.0000	0.0000	0.0000	0.0000	0.1667	0.5714	0.0000	0.0000
ufo	0.9000	0.2000	0.9000	0.2667	0.9000	0.8333	0.9000	1.0000	0.0000	0.0000

6.3 Category Performance Breakdown

Table 3 compares the average success rate (TSR) grouped dynamically by application category.

Table 3: TSR comparison grouped by task category.

Agent	Browser	Calculator	File Manager	Graphics	Multi-App	Settings	Task Manager	Text Editor
noop	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
random	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
rule	0.0000	0.2963	0.0000	0.0000	0.0000	0.3333	0.5000	0.0000
claude	0.2444	0.2222	0.2333	0.1667	0.3636	0.2222	0.0000	0.3389
omniparser	0.0870	0.2222	0.1778	0.1111	0.1364	0.0741	0.0000	0.2667
ufo	0.5722	0.8519	0.7426	0.3333	0.5909	0.6667	0.3333	0.7278

7. Discussion

7.1 Hypothesis Validation

We validated the three core hypotheses formulated for DesktopAgentBench reliability evaluation:

1. **H1: Combined chaos causes the largest degradation — NOT SUPPORTED**
   - *Evidence*: The UFO agent's Moderate combined chaos TSR is 0.2667, whereas its TSR under the isolated Fake UAC chaos variant drops to 0.0000 (complete blocker).
   - *Discussion*: The Fake UAC chaos profile completely blocks user inputs and access to the target window controls, acting as an unrecoverable single-point-of-failure. Because isolated UAC is more destructive than the combined moderate profile (which lacks UAC prompts), combined moderate chaos is not the worst-case degradation.
2. **H2: Focus-aware agents recover better than naive agents — SUPPORTED**
   - *Evidence*: The UFO agent (focus-aware UIA agent) achieved a Recovery Rate (RR) of 0.5773 and the Rule agent (focus-aware) achieved 0.0955 RR, whereas the focus-naive agents (Claude and OmniParser) achieved significantly lower recovery rates of 0.1227 and 0.0409 respectively.
   - *Discussion*: The active UIA window watcher and focus tracking policies of UFO enable it to handle window focus loss (Focus Steal TSR: 0.9000) and modal overlaps (Popup TSR: 0.8333). The coordinate-naive agents stall or fail completely in these circumstances.
3. **H3: Random agents create more latent harm than structured agents — NOT SUPPORTED**
   - *Evidence*: The Claude agent registered a Latent Harm Detection Index (LHDI) score of 0.1038, whereas the Random agent registered 0.0800 (and OmniParser registered 0.0908).

- *Discussion*: Because Claude is a structured focus-naive agent that uses fixed absolute coordinate clicks, it continues to click blindly even when focus is stolen or a window shifts. Because these clicks are targeted in rapid succession, they interact with unintended background windows, spawning processes, editing files, or altering system files. In contrast, the Random agent's clicks are spatially scattered and less likely to trigger coordinated side-effects.
- *Finding 2 (Mechanism of Latent Harm)*: The increase in Claude's LHDI compared to random baselines is driven by the fact that coordinate-naive agents issue rapid, sequential clicks to fixed screen locations based on outdated screenshots. When foreground focus shifts, these click sequences execute coherently in background applications, triggering compound unintended actions (such as opening administrative menus, editing inactive system files, or launching processes). Random agents, by contrast, execute disjointed clicks that fail to form a coherent command chain. Additionally, Claude and OmniParser score higher TSR on the `scroll_hide` disruption variant because the subset of tasks they successfully complete (specifically calculator arithmetic and basic notepad entry) do not require scrolling interaction; thus, scrollbar hiding does not interfere with their action plans.

7.2 Production Readiness

The results demonstrate that evaluating agents in pure "clean" states hides essential design deficiencies. A capable agent might successfully solve mathematical calculations in a sandboxed, static window, but fail instantly if a background notification window overlaps its canvas.

For an agent to be production-ready, it must be equipped with active perceptual loops: * **Focus Checking:** The agent must verify if its target window has active foreground focus before sending raw key sequences. * **Alert Resolution:** The agent must recognize blocking alert elements (such as notification toasts or popups) and resolve them (e.g. click "close") instead of continuing its preset action queue blindly.

DesktopAgentBench provides a valuable benchmark for industry and research by forcing developers to move beyond capabilities-oriented metrics toward robustness and recovery testing.

7.3 Limitations

- **Corpus Size:** The dev split contains 30 validated tasks, which is sufficient for reference baselines but smaller than general capability benchmarks.
- **Network Independence:** To maintain reproducible execution, tasks are kept network-independent. Consequently, browser tasks navigate local

caches or simple domains (`example.com`), omitting complex cloud auth dynamics.

- **Win32 API Bindings:** The chaos engine depends heavily on native Windows user32/kernel32 calls. Modifications in future Windows 11 updates may require updates to coordinate offset calculations or window class filters.
- **Simulated Agent Architectures:** The Claude, OmniParser, and UFO agents evaluated in this benchmark are behavioral simulations designed to model their interface control styles rather than live production integrations.
- **Single OS/Resolution Configuration:** All evaluations are performed under a single operating system environment (Windows 11) at a fixed screen resolution (1920×1080), meaning results may vary on alternative desktop configurations.

---

8. Conclusion

DesktopAgentBench addresses the need for reliability testing in GUI automation. By evaluating agents under safe, parameterizable, and reversible chaos injection, we expose failures in naive agent focus loops and action scheduling. We hope this benchmark encourages the development of GUI agents equipped with active perception and error recovery routines, moving beyond capabilities toward true production readiness.

References

- Xie, T., Zhang, S., Chen, Z., Xia, Y., & Liu, P. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Computer Control. *arXiv preprint arXiv:2404.07972*.
- Bonatti, R., et al. (2024). WindowsAgentArena: Evaluating Multi-Agent Systems on Windows Desktops. *arXiv preprint arXiv:2409.08264*.
- GUI-Robust (2025). Robustness of GUI-based Agents under Visual Anomalies and Dynamic Layouts. *arXiv preprint arXiv:2506.14477*.
- AgentHijack (2026). AgentHijack: Adversarial Prompt Injection and Hijacking in Desktop Automation. *arXiv preprint arXiv:2605.25707*.
- Zhang, C., et al. (2024). UFO: A User Interface Agent for Windows Applications. *arXiv preprint arXiv:2402.07931*.
- Lu, Y., et al. (2024). OmniParser: A Unified Visual Parser for GUI Agent Control. *arXiv preprint arXiv:2408.00203*.

Appendix

Figure/Table Captions

- **Figure 1: DesktopAgentBench System Architecture.** Central benchmark orchestrator scheduling and executing randomized, safe, and reversible chaos injection modules (window focus, rendering timing, layout, alerts) on native Windows applications during live agent execution.
- **Table 1: Aggregate Core Metrics comparison across all tasks.** Metrics computed across the entire evaluation matrix (1,400 runs per agent baseline, and 280 runs for the specialized rule baseline, totaling 7,280 runs in the evaluation matrix). The UIA-aware UFO agent shows the highest Task Success Rate (TSR) and Recovery Rate (RR).
- **Table 2: TSR breakdown across Clean and Chaos profiles.** Success rates across clean baseline, isolated disruptions, and moderate combined chaos profiles. Claude and OmniParser agents fail completely under popups and focus steals, whereas UFO handles them robustly.
- **Table 3: TSR comparison grouped by task category.** Performance breakdown grouped dynamically by application category. UFO performs consistently across categories except Graphics and Browser.

Contribution Summary

In this work, we present **DesktopAgentBench**, an evaluation harness designed to test desktop automation agent resilience under safe, native Windows chaos injection. We introduce a structured task corpus of 30 tasks across 8 categories, a parameterizable chaos spawner with 7 reversible modules, and an evaluation framework tracking 7 core metrics. We evaluate no-op, random, rule-based, UFO, Claude, and OmniParser agent profiles across a matrix of 1,400 runs per agent baseline (and 280 runs for the specialized rule baseline, totaling 7,280 runs in the evaluation matrix) under 10 distinct chaos profiles. We show that focus-aware targeting policies are the primary determinant of agent resilience in noisy desktop environments, and demonstrate that DesktopAgentBench successfully exposes and measures these vulnerabilities to track agent production readiness.